POLITECNICO MILANO 1863

ROS2 Fundamentals

Robotics Academy

02.07.2024 | Lorenzo Ortolani, Gabriel Voss

Academy Details

General Info

Evaluation final project

- Each session is divided by a theoretical part and practical session with showcased examples and hands on exercises on small tasks that you'll need to solve by your own with our supervision.

- Exercises are necessary for getting familiar with ROS2 framework and with general robotics topics.

Academy Details

General Info

Evaluation final project

Each participant of the academy will solve a final project that will be evaluated based on the following criteria:

- Quality of code and code reproducibility
- How your robotic system agent will react in a realistic simulation setting

Overall quality of your GitHub repository:

- Documentation quality (e.g. well structured Readme.md with emphasis on your technical reasoning steps)
- Structure of your ROS2 created package for running everything on the robot

Repository of the Academy

Here you can find the official repository of the robotics academy with all the material that we will attach. Follow the reported instructions in the Readme.md.
https://github.com/Relo02/Robotics_academy

In order to clone the repo, run the following command:

```
$ git clone —-recursive https://github.com/Relo02/Robotics_academy
```

Contents

01

Ros2 Fundamentals

- What is ROS2?
- Docker environment
- ROS2 Concepts

02

Control Systems and State Estimation

- Control Systems in ROS2
- State Estimation: EKF vs UKF, Particle Filters and Sensor Fusion
- VO/VIO

03

SLAM

- ROS2: Slam_toolbox
- Practical example on a robot in Gazebo and Rviz
- Custom Localization/mappi-ng examples

04

Path Planning and Navigation

- A* global planner
- RRT/RRT*
- Optimization based planning
- Examples of Nav2 integration

05

Physical AI – New Robotics frontiers

- Deep Learning based Robotic Perception and locomotion
- Imitation Learning and VLA models
- Few real word showcase

ROS2 Fundamentals

01

What is ROS2?

01.1

What is ROS2?

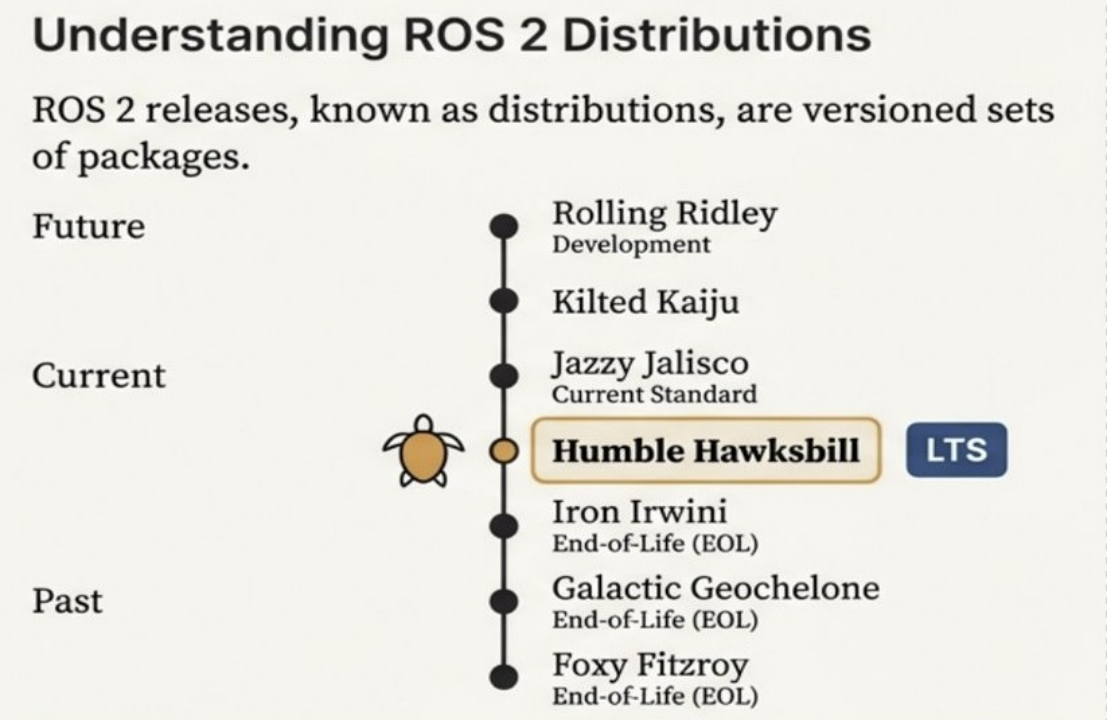
ROS 2 (Robot Operating System 2) is an open-source middleware framework for robotics software development.

Key ideas:

- Provides standard tools and libraries to build robot applications
- Enables modular and distributed systems
- Simplifies communication between sensors, algorithms, and actuators
- Supports real-time and multi-robot systems
- Runs on Linux, Windows, and macOS

What is ROS2?

We are going to use ROS2 humble distribution, since it’s one of the most stable and widely used ROS2 distribution in the community.



Docker environment

01.2

Docker environment

- **What is Docker?**

Docker is a virtualized operating system, which delivers software packages in containers. Docker automates the deployment of applications within lightweight containers, enabling them to run consistently across different computing environments.

- **Why we need to use Docker?**

Docker is essential for letting you run different types of dependencies that in principle are not compatible for all operating systems. So for the seek of simplicity, we will all work in the same environment such that everyone has the same framework setup.

After you have installed docker, start to follow the Robotics Academy instructions provided by the Readme.md

Guide for installing docker engine:
https://docs.docker.com/engine/install/

ROS2 Concepts

01.3

ROS2 BASICS

⚙️ Node

The fundamental unit of computation. A single self-contained process — e.g. a camera driver, a path planner, a sensor filter.

📦 Package

Primary organizer of ROS 2 code. Contains nodes, launch files, config files, and a package.xml manifest defining dependencies.

🗂️ Workspace

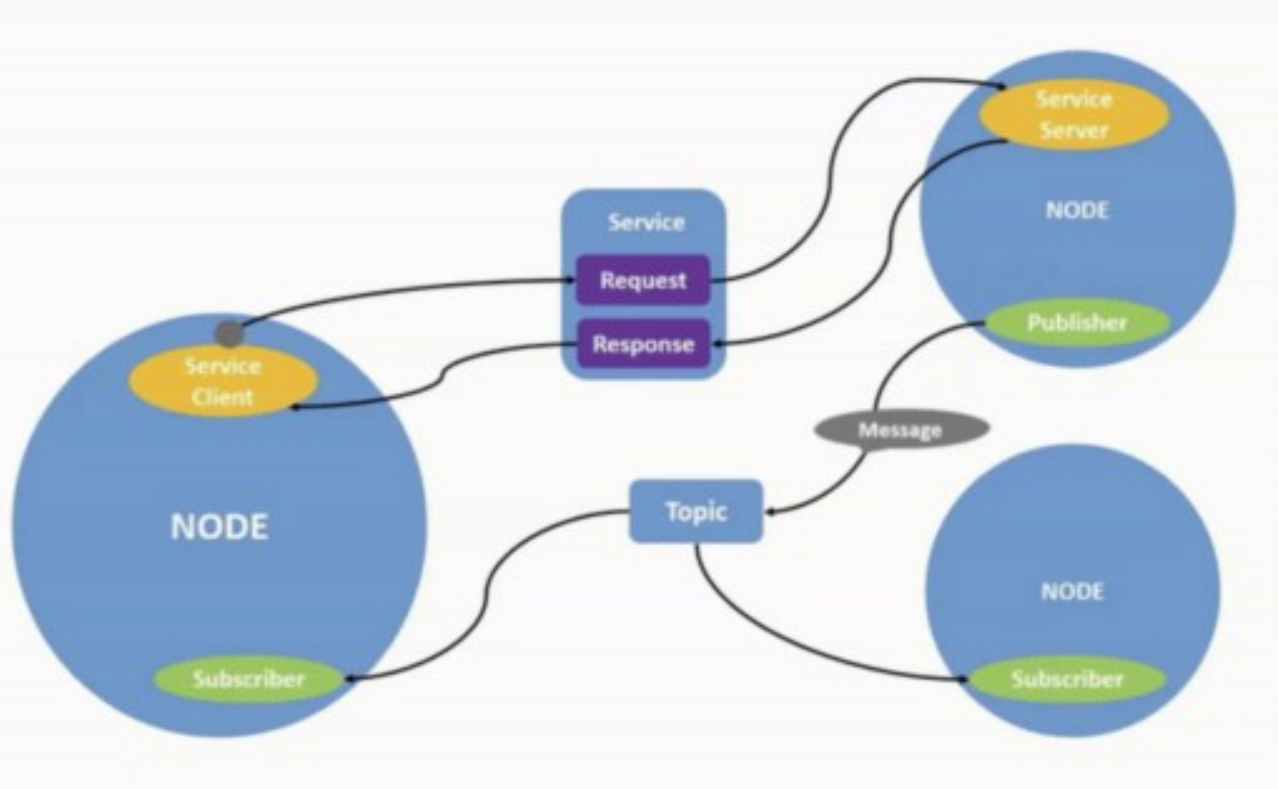
A directory holding a collection of packages. Build all packages at once with colcon build.

workspace structure

```
~/ros2_ws/                      ← Your workspace root
  src/                            ← Source packages go here
    my_robot_pkg/
      package.xml                 ← Dependencies & metadata
      CMakeLists.txt              ← Build instructions
      setup.py                    ← (Python packages)
      my_robot_pkg/
        my_node.py                ← Your node code
      launch/
        robot.launch.py           ← Launch files
      config/
        params.yaml               ← Parameters
  build/   install/   log/        ← Generated by colcon build
```

Understanding Nodes

Each node in ROS should be responsible for a single, modular purpose, e.g. controlling the wheel motors or publishing the sensor data from a laser range-finder. Each node can send and receive data from other nodes via topics, services, actions, or parameters



Understanding Nodes

- Single-purpose, executable program
- Individually compiled, executed, and managed
- Organized in packages

Run a node:

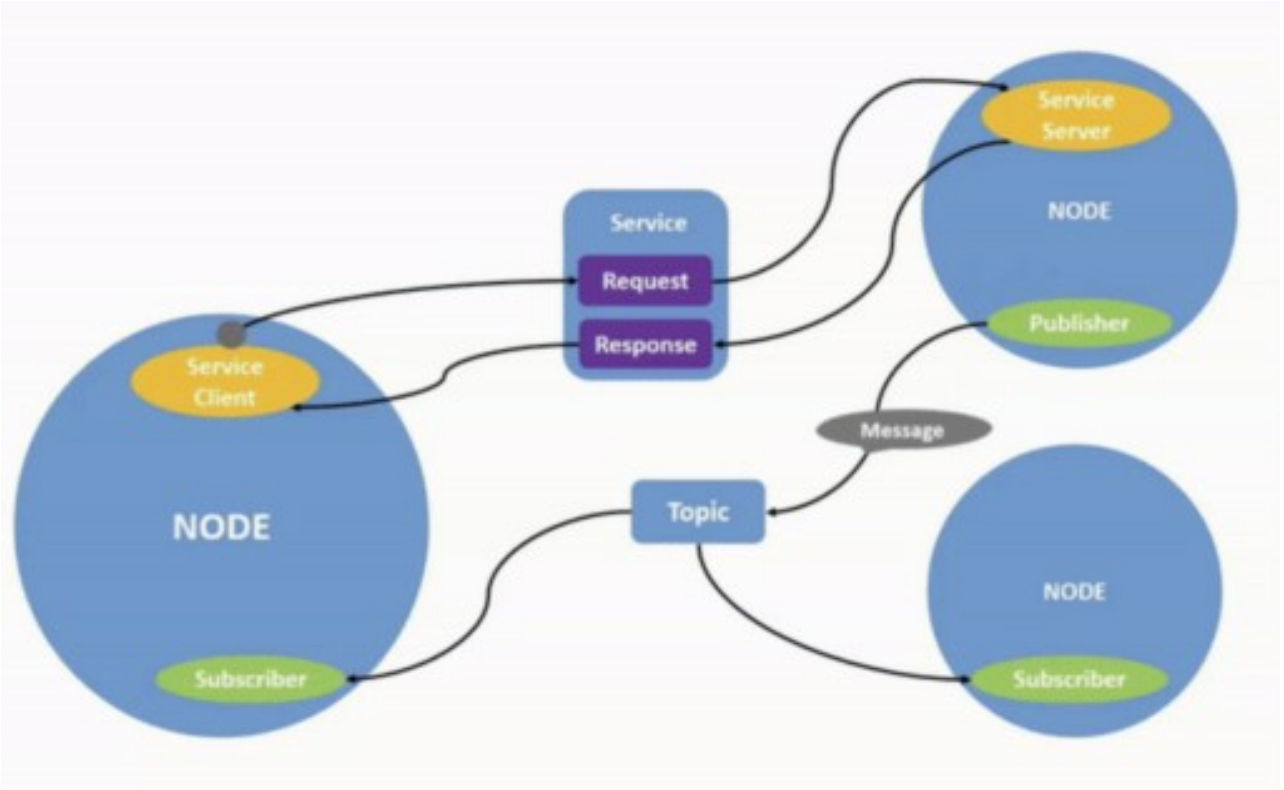
```
$ ros2 run package_name node_name
```

See active nodes with

```
$ ros2 node list
```

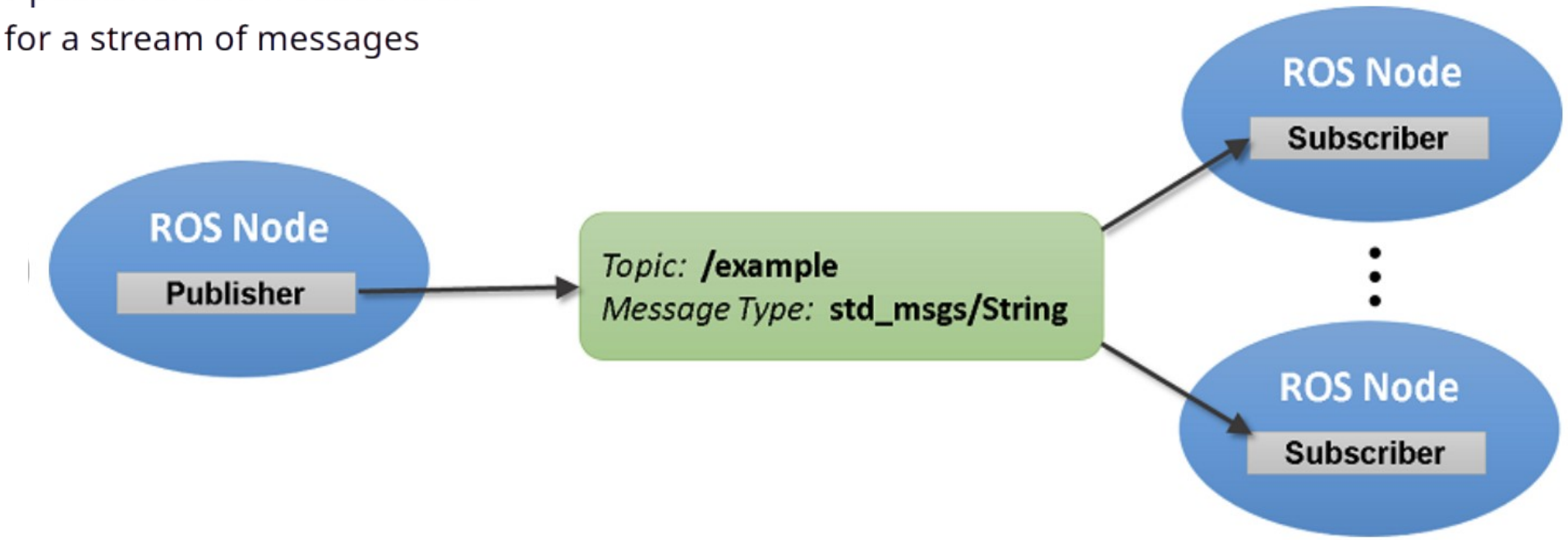
Retrieve information about a node with

```
$ ros2 node info node_name
```

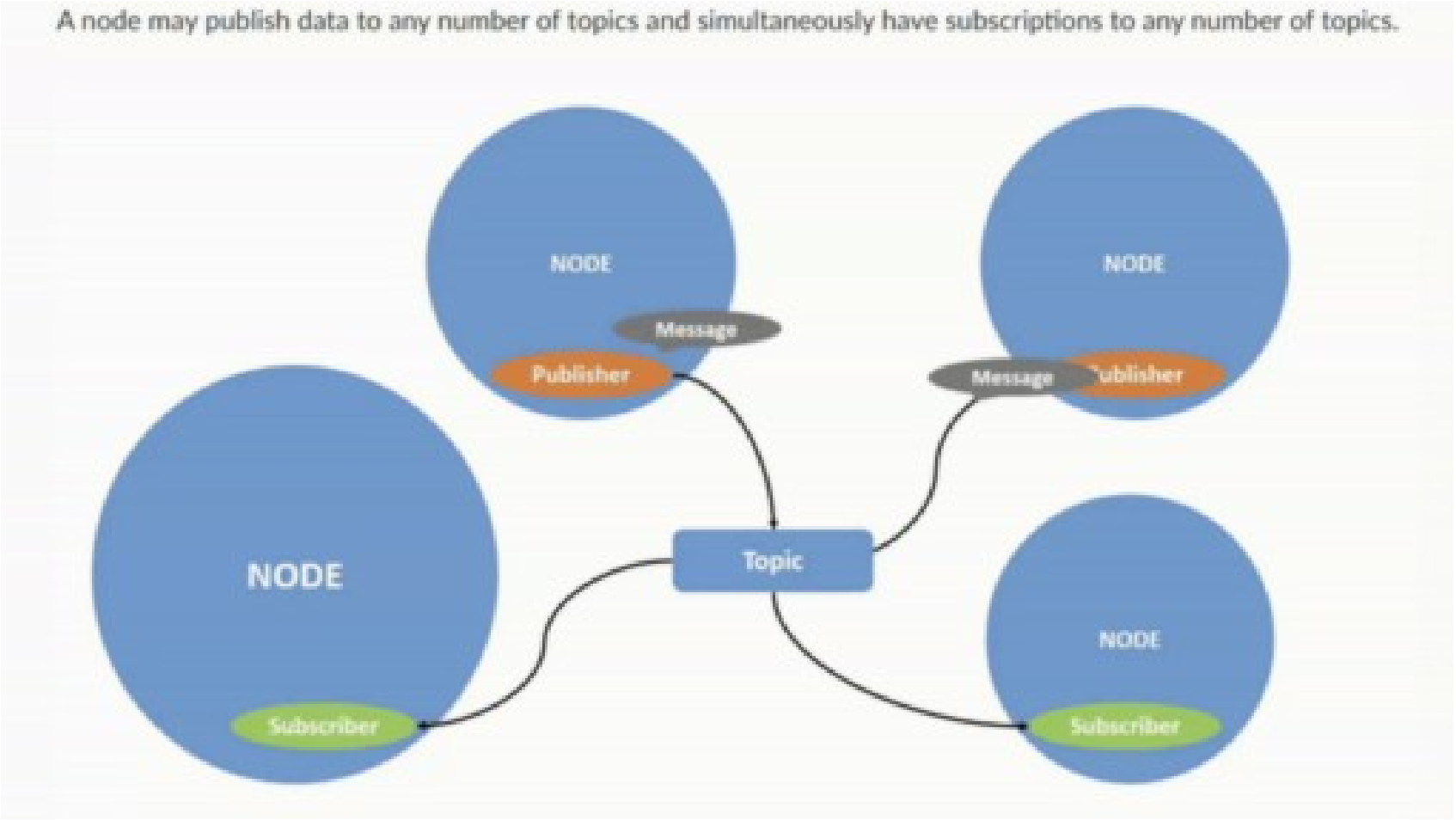


Understanding Topics

- Nodes communicate over topics
  - Nodes can publish or subscribe to a topic
  - Typically, 1 publisher and *n* subscribers
- Topic is a name for a stream of messages



Understanding Topics



Understanding Topics

Few examples of commads that you can run in ROS for observing the topics status

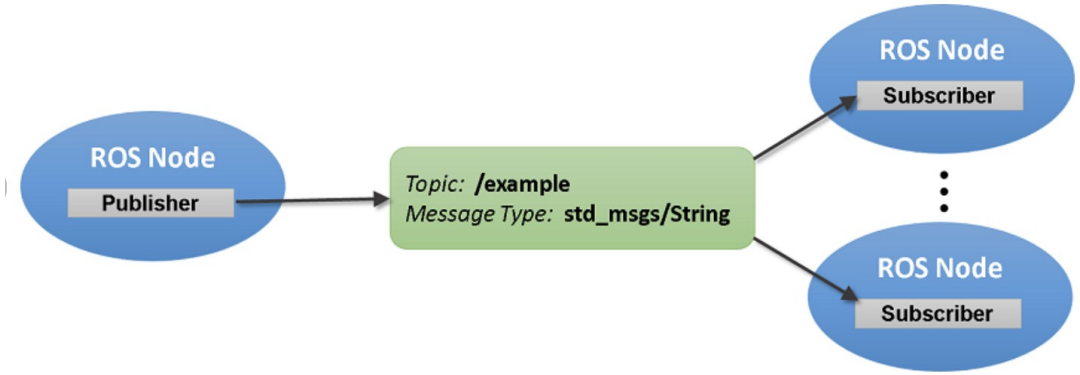
bash — ros2 topic commands

```bash
$ros2 topic list
  # List all active topics
$ros2 topic info/chatter
  # Show publisher/subscriber count & type
$ros2 topic echo/chatter
  # Print messages arriving on a topic
$ros2 topic pub/chatter std_msgs/String
"data: 'hello'"
  # Publish a one-shot message manually
$ros2 topic hz/scan
  # Measure publishing frequency
$ros2 topic bw/camera/image_raw
  # Measure bandwidth usage
```

Messages

- In ROS2 **messages** have their own **interface**, which are described in a `.msg` file or `.srv`. These files are built by `colcon` into libraries that can be imported by your Python programs.

- In most practical robotics applications, is a good practice to use the already existing ROS2 interfaces, instead of creating new once that can show the same purpose.



Listing all the ROS2 available interfaces:

```
$ ros2 interface packages
```

```
geometry_msgs
lifecycle_msgs
logging_demo
map_msgs
nav_msgs
pcl_msgs
pendulum_msgs
rcl_interfaces
rmw_dds_common
rosbag2_interfaces
rosgraph_msgs
sensors_msgs
```

Types

- Each interface has its own message with an associated **type**.

Listing few messages types from some interfaces:

```
$ ros2 interface package example_interfaces
```

```
example_interfaces/msg/String
example_interfaces/srv/AddTwoInts
example_interfaces/srv/SetBool
example_interfaces/msg/UInt8
example_interfaces/msg/Int64MultiArray
example_interfaces/msg/Byte
example_interfaces/msg/Float32
example_interfaces/msg/Int64
example_interfaces/msg/UInt32MultiArray
example_interfaces/msg/Int32MultiArray
example_interfaces/msg/Empty
```

Services

- In ROS2, services provide **synchronous** communication between nodes using a request–response model.

- A **client** node sends a request, and a **server** node processes it and returns a response.

Characteristics:
- Synchronous interaction
- Used for short, immediate operations
- Based on request/response messages
- The client waits for the response

When to use services:

Services are used when a single action needs a direct answer.
Typical examples:
- Reset a robot controller
- Request the robot position
- Start or stop a subsystem
- Retrieve configuration parameters

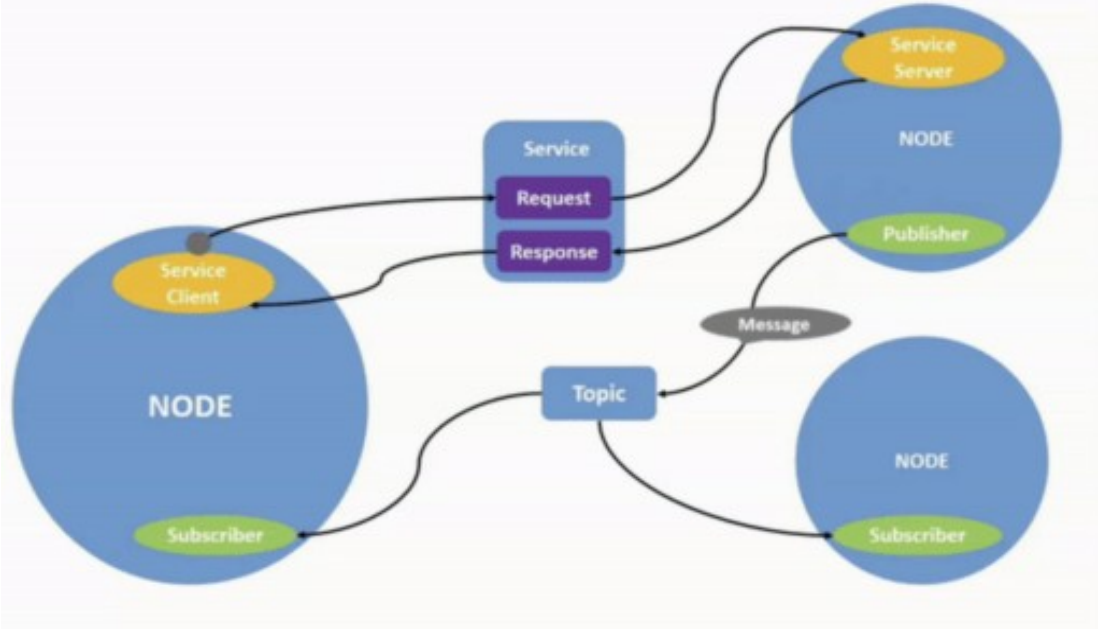
Example:

```
Localization Node → request → get_map
Map Server → response → map data
```

Services

Localization Node requests to get the data about the map structure:

```
Localization Node → request → get_map
Map Server → response → map data
```



Services

Useful ROS2 services commands:

```
$ ros2 service list              # list all the available services
$ ros2 service type service_name # check the service type
$ ros2 service call service_name # call a particular service from the server
```

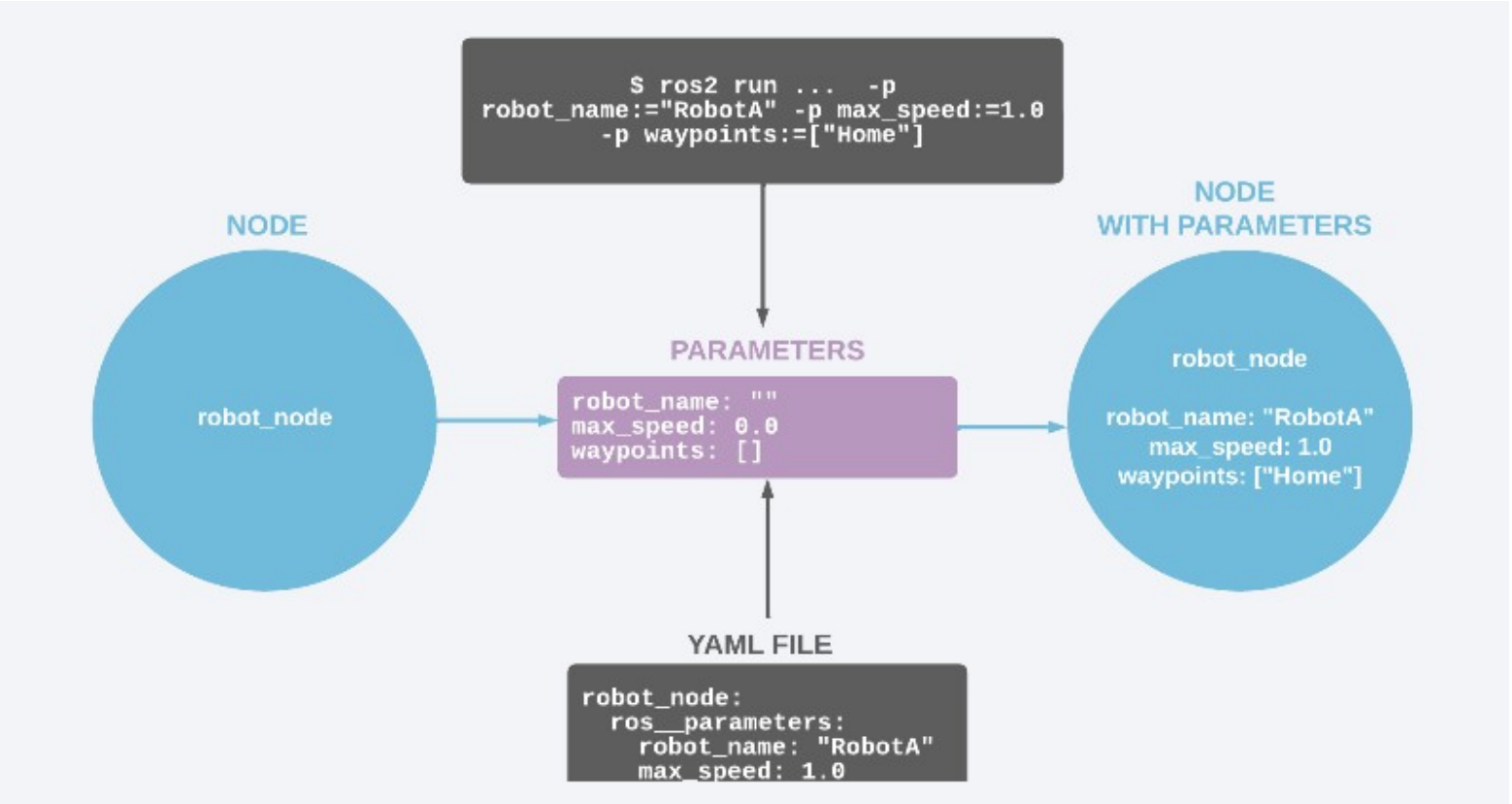
Services vs Topics:

Feature	Topics	Services
Communication	asynchronous	synchronous
Data flow	continuous	request-response
Use case	sensor streams	commands / queries

Parameters

- Parameters are configuration values of a node. In practice, they are node settings. A node can store parameters as integers, strings, floats, booleans, etc...

- In ROS2, each node has it's own parameter.



Parameters

To display the type and the current value of a parameter, use the command:

```
$ ros2 param get <node_name> <param_name>
```

Expected output if the parameter is an integer:

```
Integer value is: 86
```

Ros2 param set: is used for changing the parameter's value at runtime, using the following command:

```
$ ros2 param set <node_name> <parameter_name> <value>
```

For more information, visit the official ros2 documentation:
https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Parameters/Understanding-ROS2-Parameters.html

Launch files

- Launch files allows you to run multiple nodes in once.
- They can be easily integrated when you have a complex package structure with multiple executables (so nodes) that require to be launched at the same time.

```python
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            executable='mimic',
            name='mimic',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )
    ])
```

Launch files

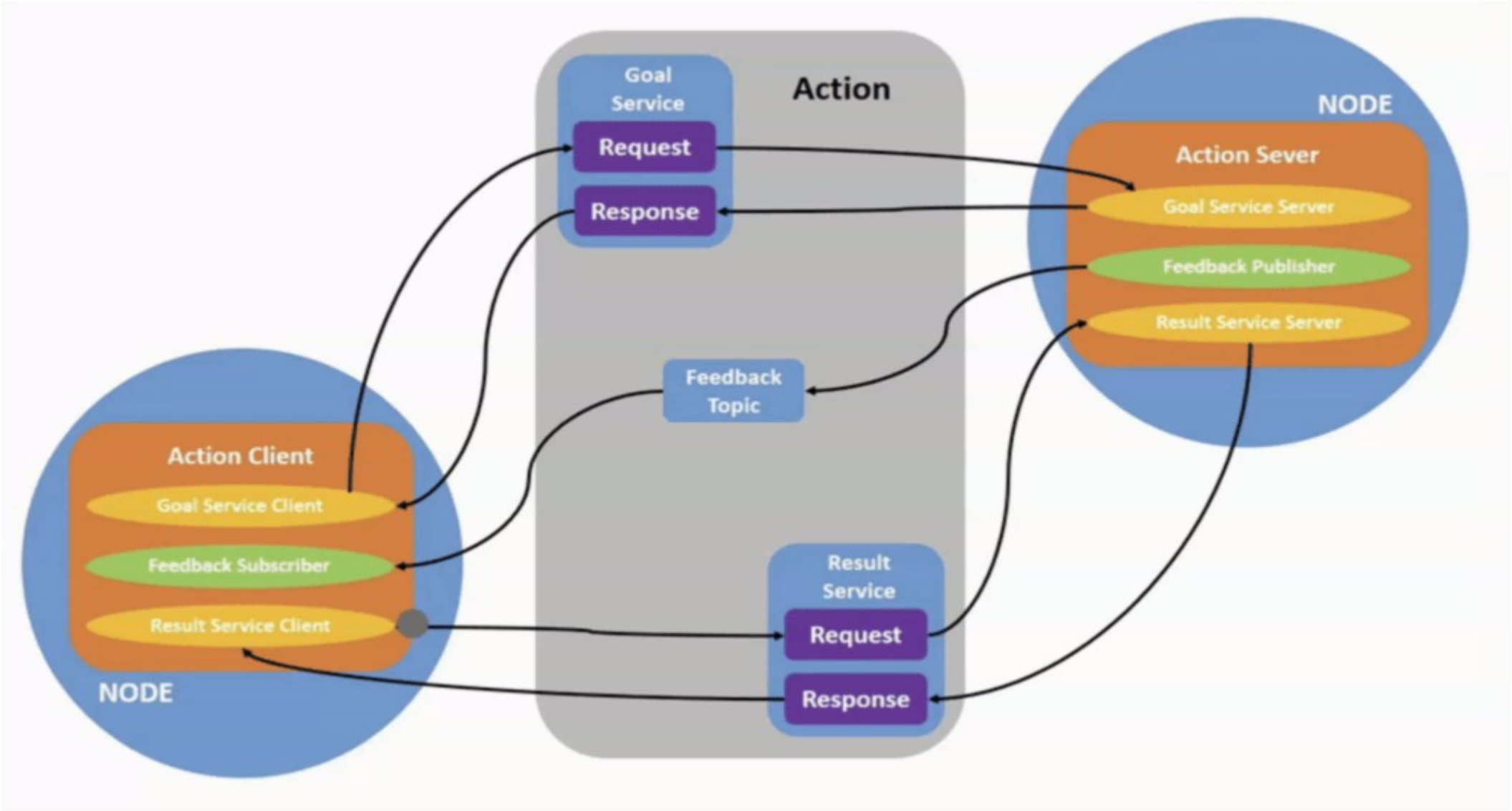
We will be focusing more on Python packages formatting. Therefore, if you have created a new launch file, you need to specify in **setup.py** the directory of your launch file.

```
import os
from glob import glob
from setuptools import setup

package_name = 'py_launch_example'

setup(
    # Other parameters ...
    data_files=[
        # ... Other data files
        # Include all launch files.
        (os.path.join('share', package_name, 'launch'),
glob(os.path.join('launch', '*launch.[pxy][yma]*')))
    ]
)
```

Actions


Goal
Service
Request
Response
Action
NODE
Action Sever
Goal Service Server
Feedback Publisher
Result Service Server
Feedback
Topic
Action Client
Goal Service Client
Feedback Subscriber
Result Service Client
NODE
Result
Service
Request
Response

Actions

- Actions are one of the communication types in ROS2 and are intended for long running tasks. They consist of three parts: a goal, feedback, and result.
- Actions are built on topics and services. Their functionality is like services, except actions are preemptable (you can cancel them while executing). They also provide steady feedback, as opposed to services which return a single response.
- Actions use a client-server model, similar to the publisher-subscriber model. An “action client” node sends a goal to an “action server” node that acknowledges the goal and returns a stream of feedback and a result.

Check available actions command:

```
$ ros2 action list -t
```

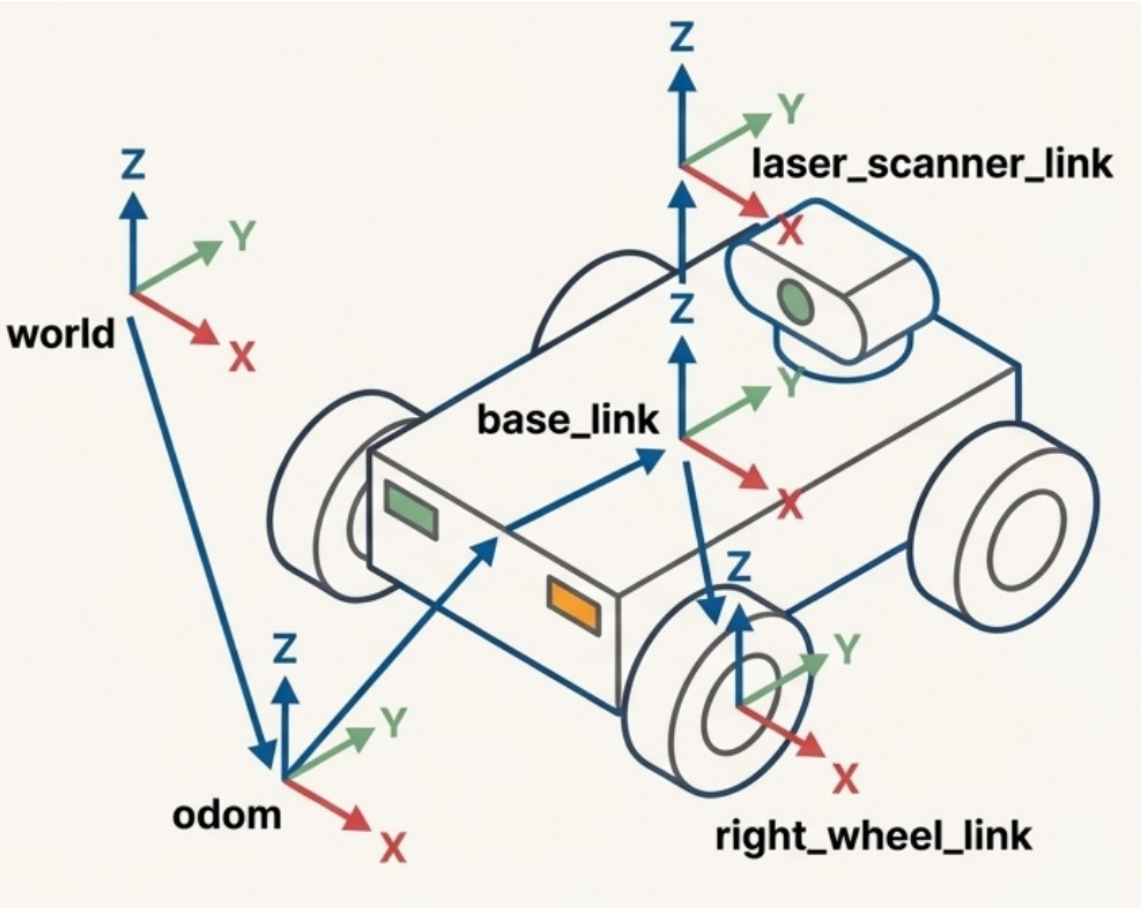
TF2

A robot is made of many parts (base, wheels, arm,
sensors), each with its own coordinate system. How do you relate a sensor reading in the "laser_frame" to the robot's "base_link" frame?

The Solution: The tf2 Transform Library

tf2 is a core ROS 2 library that keeps track of the relationships between all coordinate frames in a system over time.

- A broadcaster node publishes the geometric relationship (transforms) between frames.
- A listener node can then query tf2 to compute the transformation between any two frames at any given time.

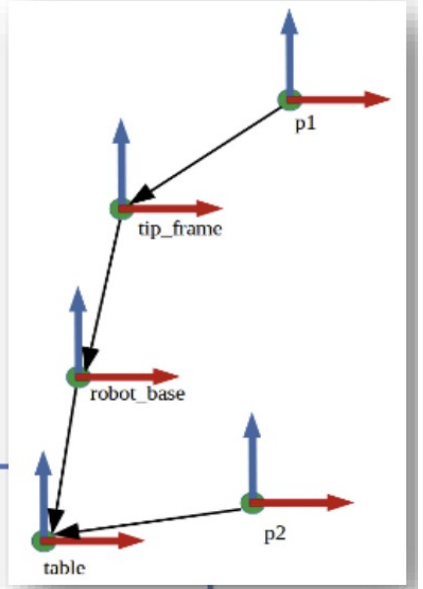


TF2 – Additional tutorial materials

Examples linked into the repository:
https://github.com/Relo02/Robotics_academy/tree/main/tutorials/tf2

URDF

- Unified Robot Description Format (URDF) is an XML format for representing a robot model.
- It enable to describe kinematic, visual and dynamic properties of a manipulator.
- **Xacro** is an XML macro language: enable construction of shorter and more readable XML files by using macros that expand to larger XML expressions.

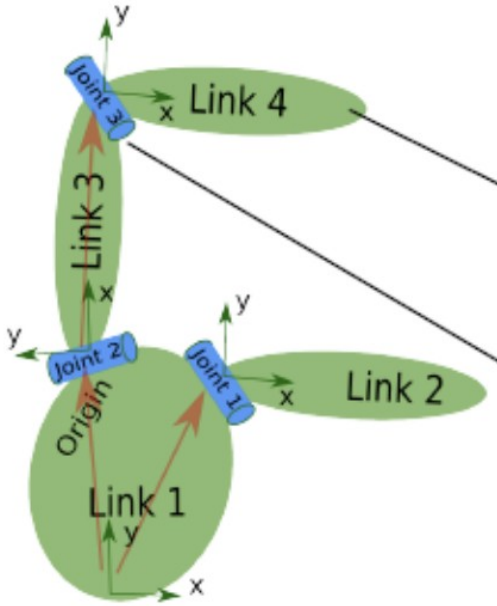



```
<robot name="test_robot">
  <link name="link1" />
  <link name="link2" />
  <link name="link3" />
  <link name="link4" />

  <joint name="joint1" type="continuous">
    <parent link="link1"/>
    <child link="link2"/>
  </joint>
  <joint name="joint2" type="continuous">
    <parent link="link1"/>
    <child link="link3"/>
  </joint>
  <joint name="joint3" type="continuous">
    <parent link="link3"/>
    <child link="link4"/>
  </joint>
</robot>
```

URDF – Simple Example

- Description consists of a set of *link* elements and a set of *joint* elements
- Joints connects the links together



robot.urdf

```
<robot name="robot">
  <link> ... </link>
  <link> ... </link>
  <link> ... </link>

  <joint> .... </joint>
  <joint> .... </joint>
  <joint> .... </joint>
</robot>
```

```
<link name="link_name">
  <visual>
    <geometry>
      <mesh filename="mesh.dae"/>
    </geometry>
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="10"/>
    <inertia ixx="0.4" ixy="0.0" …/>
  </inertial>
</link>
```

```
<joint name="joint_name" type="revolute">
  <axis xyz="0 0 1"/>
  <limit effort="1000.0" upper="0.548" … />
  <origin rpy="0 0 0" xyz="0.2 0.01 0"/>
  <parent link="parent_link_name"/>
  <child link="child_link_name"/>
</joint>
```

URDF – Additional tutorial materials

Practical tutorials examples materials:
https://github.com/Relo02/Robotics_academy/tree/main/tutorials/urdf

Thank you for your attention